

# Roots That Are Always There

The virtual roots of a real polynomial, machine-checked in Lean 4

Carles Marín\*

Independent researcher  
karlesmarin@gmail.com

June 2026

## Abstract

A real polynomial of degree  $d$  may have fewer than  $d$  real roots, but it always has exactly  $d$  *virtual roots*: continuous, semialgebraic substitutes  $\rho_{d,1}(p) \leq \dots \leq \rho_{d,d}(p)$  that coincide with the real roots when these exist and otherwise sit where the polynomial comes closest to vanishing. They were introduced by González-Vega, Lombardi and Mahé [1] and are the natural carrier of the Budan–Fourier count [2]. I report a complete, `sorry`-free formalization in Lean 4 over Mathlib of their *structural theory*: the recursive construction by a choice operator  $\mathcal{R}_d$  that minimises  $|p|$  on each interval where  $p'$  keeps a constant sign; the fundamental count, that there are exactly  $d$  of them (`vroots_length`); that they come out sorted and inside the bracketing interval (`vroots_isChain`, `vroots_subset_Icc`); that  $\mathcal{R}_d$  returns an actual root wherever  $p$  has one (`R_eval_eq_zero_of_le_zero_le`); and the *Rolle interlacing*

$$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p),$$

the statement that each virtual root of  $p'$  is wedged between two consecutive virtual roots of  $p$  (`vroots_interlacing`). To the best of my knowledge—based on searches in June 2026 of Mathlib, the Coq `mathcomp-real-closed` library, the Isabelle AFP, and HOL Light (Section 8)—virtual roots had not been formalized in any interactive theorem prover; this is their first development. What is deliberately *not* here is the exact Budan–Fourier count, that the number of virtual roots in  $(a, b]$  equals the sign-variation drop  $V(a) - V(b)$ : that theorem fuses this construction with a second, independent one (the `fourierVar` of the companion Budan–Fourier formalization [18]) and is the explicit sequel, named honestly in Section 8. Every theorem here depends only on the three standard axioms `propext`, `Classical.choice`, `Quot.sound`.

*What you will find here.* A short guided tour, built as a string of beads: each section ends with the question the next answers. The first asks what a virtual root *is*, and why there are always exactly  $d$  of them (Section 2); the second, how to build them so a machine will accept the construction (Section 3); the third, where the formalization genuinely resists (Section 4); the fourth, what the certified theory gives—the Rolle interlacing, and the road to the exact count (Section 5). The load-bearing lemmas are in one table (Section 7); the honest limits, including the deferred count, are in Section 8; and the close asks the questions worth carrying forward. If you read only two things, read the interlacing (Theorem 1) and Figure 1.

---

\*Formalized with the assistance of an AI system (Claude, Anthropic); the mathematics is classical, and every claim, scope statement, and limitation is the author’s responsibility. The boundary of the contribution—in particular what is *not* yet formalized—is set out plainly in Section 8, and the Lean kernel, not the assistant, is what certifies the proofs.

# 1 The question that begins it

*History bead.* Budan [4] and Fourier [5] could bound the number of real roots of a polynomial in an interval by a difference of sign counts, but the bound is only an inequality: the surplus, always even, is the price of the roots that fail to be real. For two centuries that surplus was an error term. In 1998 González-Vega, Lombardi and Mahé [1] turned it into an object. They observed that a degree- $d$  polynomial, real-rooted or not, carries  $d$  canonical real numbers—the *virtual roots*—that vary continuously with the coefficients, restrict to the genuine roots when those are present, and account *exactly* for the Budan–Fourier count. Coste, Lajous, Lombardi and Roy later made the count an equality through them [2].

The picture to keep is the simplest non-trivial one. The polynomial  $p = x^2 + 1$  has no real root, yet its graph dips to a single closest approach at  $x = 0$ . Virtual roots see that approach:  $p$  has the double virtual root  $\rho_{2,1} = \rho_{2,2} = 0$ . Where a real root would be, there is a virtual one; where two conjugate roots hide, the two virtual roots collide at the foot of the dip (Figure 1). Nothing is invented—the construction is explicit and semialgebraic—and nothing is lost: the count is always  $d$ .

This paper formalizes the part of that story that is structural: the construction, the count, the order, and the interlacing. The arithmetic equality with the sign-variation drop—the part that earns the name “Budan–Fourier”—is the sequel, and Section 8 says exactly why it is held back. What begins it is the question: *what is a virtual root, concretely enough that a proof assistant will accept it, and why are there always  $d$ ?*

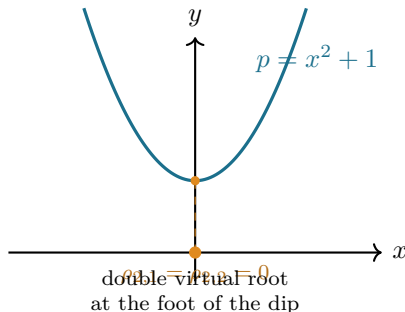


Figure 1: Roots that are always there. The polynomial  $x^2 + 1$  has *no* real root, but a double virtual root at  $x = 0$ , where the graph comes closest to the axis. A degree-2 polynomial has two virtual roots whatever its discriminant; here they coincide. In Lean, on a bracketing interval, the construction returns the list  $[0, 0]$ .

## 2 Q1: what is a virtual root, and why exactly $d$ ?

*The idea in one line.* Between two consecutive critical points of  $p$ —two consecutive roots of  $p'$ —the derivative keeps a constant sign, so  $p$  is strictly monotone there and  $|p|$  has a single minimum. The point where that minimum is attained is a virtual root: the unique root of  $p$  in the cell if  $p$  changes sign across it, and otherwise the cell endpoint where  $p$  is smallest. The critical points of  $p$  are the virtual roots of  $p'$ , so the construction recurses on the derivative, exactly as Rolle’s theorem [6] interleaves the roots of  $p$  and  $p'$ .

*Why exactly  $d$ .* A degree- $d$  polynomial has a degree- $(d-1)$  derivative, hence—by induction— $d-1$  virtual roots of  $p'$ ; these cut the line into  $d$  cells, and each cell contributes one virtual root of  $p$ . The count never degenerates: it is the degree, not the number of real roots. This is the first theorem, and the cleanest:

$$\# \{ \text{virtual roots of } p \} = \deg p.$$

In Lean this is `roots_length`: the list `roots lo hi p` of virtual roots has length `p.natDegree`. Real roots are at most  $d$  (and can be far fewer, or none); **virtual roots are always exactly  $d$** . The gap between the two counts is precisely the even Budan–Fourier surplus, now realized as collisions of virtual roots away from the axis.

The bead handed on: this is a clean recursive recipe on paper—*minimise  $|p|$  on each  $p'$ -monotone cell*—but “the point where  $|p|$  is minimised” is a choice, and “the cells” are cut by an object defined by the same recursion. How does one hand that to a machine?

### 3 Q2: how to build them so a machine accepts it

*The bracketing interval.* The cells of  $p$  are cut by the virtual roots of  $p'$  together with two outer fences. Rather than work on all of  $\mathbb{R}$  from the start, I fix a closed interval  $[lo, hi]$  and build the virtual roots inside it; taking  $lo, hi$  beyond every root of the derivative tower (a Cauchy bound) recovers the global object, and that last step is discussed in Section 8. Working on  $[lo, hi]$  keeps every quantity a genuine real number and lets the recursion reuse one choice operator verbatim.

*The choice operator  $\mathcal{R}_d$ .* On a closed interval  $[a, b]$ , the continuous function  $|p|$  attains a minimum (the interval is compact); call a minimiser  $\mathcal{R}_d(a, b, p)$ . In Lean this is `R`, defined by choosing a minimiser whose existence is `exists_isMinOn_abs_eval`. Two facts about it carry the whole theory. First, it **lands in the interval**: `R_mem` says  $\mathcal{R}_d(a, b, p) \in [a, b]$ . Second, it **captures actual roots**: if  $p(a) \leq 0 \leq p(b)$  (or the reverse), the intermediate value theorem gives a root, the minimum of  $|p|$  is then 0, and the minimiser is that root—`R_eval_eq_zero_of_le_zero_le`. So on a cell where  $p$  changes sign,  $\mathcal{R}_d$  is the genuine root; where  $p$  does not, it is the nearer endpoint. That is the precise sense in which virtual roots extend the real ones.

*The recursion.* The virtual roots are then assembled by listing the breakpoints

$$\text{bps} = lo :: (\text{virtual roots of } p') ++ [hi],$$

and applying  $\mathcal{R}_d$  to each consecutive pair: `roots lo hi p = zipWith ( $\mathcal{R}_d$ ) p bps bps.tail`. A degree-0 polynomial has no virtual roots; otherwise the recursion descends to  $p'$ , terminating because  $\deg p' < \deg p$  (`natDegree_derivative_lt`). The well-founded definition and the length theorem are the content of `roots` and `roots_length` (Figure 2).

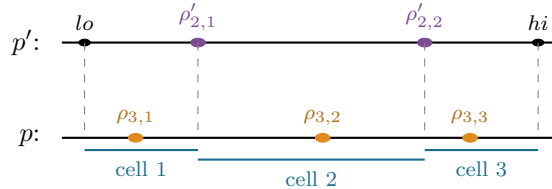


Figure 2: The recursion, for  $\deg p = 3$ . The two virtual roots of  $p'$  (plum) together with the fences  $lo, hi$  cut  $[lo, hi]$  into three cells; on each cell  $p'$  keeps a constant sign, so  $\mathcal{R}_d$  returns one virtual root of  $p$  (amber). Each  $\rho_{3,j}$  lies in its own cell—this is what makes them come out sorted and interlaced with the  $\rho'_{2,j}$ .

## 4 Q3: where the formalization resists

A *noncomputable choice, used honestly*.  $\mathcal{R}_d$  is a *chosen* minimiser, so  $\mathbf{R}$  is **noncomputable** and the development leans on `Classical.choice`. That is sound and standard, but it means the virtual roots are specified, not computed: the theorems are about *a* minimiser, and what makes them unambiguous is monotonicity. The two *monotonicity bricks* shared with the companion Budan–Fourier work—`eval_strictMonoOn_of_derivative_pos` and its negative twin—say that where  $p'$  keeps a constant nonzero sign,  $p$  is strictly monotone, so  $|p|$  has a *single* well and the minimiser is unique. The formal development needs only existence and the two facts `R_mem / R_eval_eq_zero`; uniqueness is what licenses the informal picture.

*The breakpoints are a chain, and that is the crux*. Everything downstream rests on one invariant: the augmented breakpoint list `lo :: (vroots lo hi p') ++ [hi]` is *sorted*, and every virtual root lies in  $[lo, hi]$ . The two halves are mutually dependent—a list is sorted only if its members are in range, and  $\mathcal{R}_d$  lands in range only between sorted breakpoints—so they are proved together, by a single induction down the derivative tower (`vroots_chain_mem`). The sortedness is the engine: because  $\mathcal{R}_d(\text{bps}[r], \text{bps}[r+1])$  lies in  $[\text{bps}[r], \text{bps}[r+1]]$  and consecutive breakpoints are ordered, consecutive virtual roots are ordered too (`isChain_zipWith_R`). This is Rolle’s theorem in list form, and it is the part the machine would not let me wave away.

*Indexing, and the small traps*. Turning “each virtual root sits between its breakpoints” into a statement about `getElem` required care that a hand proof hides. Rewriting a proof-carrying indexed access (`l[r]` drags a proof  $r < l.length$ ) is not a plain `rw`; it goes through `simp` with the index lemmas, relying on proof irrelevance to reconcile the bounds. And a destructuring `rcases` on an equation  $w = lo$  silently *substitutes* `lo` away in that branch—a one-character fix (`le_refl` for `le_refl lo`), but the kind of thing only a kernel catches. These are not deep, but they are exactly the hand-waves a proof assistant refuses.

The bead handed on: the chain and the per-index localization are proved; *what do they give, and what stays out of reach for now?*

## 5 Q4: what the certified theory gives

*The Rolle interlacing*. The headline consequence is the interlacing of the virtual roots of  $p$  with those of  $p'$ :

**Theorem 1** (Rolle interlacing of virtual roots; `vroots_interlacing`). *For  $lo \leq hi$  and each index  $r$  in range,*

$$\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p),$$

*i.e.  $(vroots\ lo\ hi\ p)[r] \leq (vroots\ lo\ hi\ (derivative\ p))[r] \leq (vroots\ lo\ hi\ p)[r+1]$ .*

The proof is two applications of the localization `vroots_getElem_mem`—each virtual root of  $p$  lies between two breakpoints—together with the identity that the interior breakpoints *are* the virtual roots of  $p'$ . It is the discrete shadow of Rolle’s theorem, and it places the virtual-root family on the same footing as the interlacing families that run through the real-rootedness literature—Newton’s inequalities, the Heilmann–Lieb matching polynomial—where a sequence and its derivative interleave. That this now holds for a family defined even when no real roots exist is the point.

*What it is the road to*. The interlacing is the structural half of the Budan–Fourier story. The arithmetic half—that the number of virtual roots in a half-open interval  $(a, b]$  equals the sign-variation drop  $V(a) - V(b)$  of the derivative tower [2]—is the equality that the classical Budan–Fourier inequality [18] only bounds. It is deliberately not proved here, for an honest reason explained

next: it fuses this construction with a second, independently recursive one. The structural theory is complete; the count is the sequel.

## 6 What a certified virtual-root theory enables

A machine-checked virtual-root family is the missing middle term between two things Mathlib already has and one it does not. It has Descartes’ rule (the coefficient sign count) and, in the companion work, the Budan–Fourier *inequality*; it does not have the object that would make that inequality an equality. Virtual roots are that object. Downstream, they are the right input to a certified real-root *isolation* procedure in the line of Vincent, Collins and Akritas [10, 9, 8] and the sign-variation theory of Basu–Pollack–Roy [11]—the count is exact, so a subdivision method reading  $V(a) - V(b)$  on each half has, in principle, a verified guarantee once the count theorem is added. And because the family is defined for every polynomial, real-rooted or not, it connects the interval methods to the algebra of the discriminant: a pair of virtual roots collides exactly when two real roots are about to become complex, which is where the even Budan–Fourier surplus comes from. The structural theory formalized here is the foundation all of that stands on.

## 7 The building blocks

Table 1 gathers the load-bearing results, all `sorry-free` in `VirtualRoots.lean`; Table 2 measures the development. The dependency is short: the monotonicity bricks give the existence and the two properties of  $\mathcal{R}_d$ ; the combined induction `vroots_chain_mem` proves sortedness and range together; and the per-index localization gives the interlacing.

| Lean name                                   | Statement                                                         | Status             |
|---------------------------------------------|-------------------------------------------------------------------|--------------------|
| <code>R_mem</code>                          | $\mathcal{R}_d(a, b, p) \in [a, b]$                               | proved             |
| <code>R_eval_eq_zero_of_le_zero_le</code>   | $p(a) \leq 0 \leq p(b) \Rightarrow p(\mathcal{R}_d(a, b, p)) = 0$ | proved             |
| <code>vroots_length</code>                  | $\#\{\text{virtual roots of } p\} = \deg p$                       | proved             |
| <code>vroots_isChain</code>                 | $lo :: (\text{vroots } p) ++ [hi]$ is sorted                      | proved             |
| <code>vroots_subset_Icc</code>              | every virtual root lies in $[lo, hi]$                             | proved             |
| <code>vroots_getElem_mem</code>             | $\text{bps}[r] \leq \rho_{d,r}(p) \leq \text{bps}[r+1]$           | proved             |
| <code>vroots_interlacing</code>             | $\rho_{d,r}(p) \leq \rho_{d-1,r}(p') \leq \rho_{d,r+1}(p)$        | proved             |
| <code>card_virtualRoot_eq_fourierVar</code> | $\# \text{ in } (a, b] = V(a) - V(b)$                             | <i>future (§8)</i> |

Table 1: The structural theory of virtual roots, all `sorry-free` in `VirtualRoots.lean` (Lean 4 / Mathlib). `#print axioms` on `vroots_interlacing`, `vroots_isChain`, `vroots_getElem_mem` reports only `propext`, `Classical.choice`, `Quot.sound`. The last row is the named sequel, not part of this development.

| Quantity                      | Value                                                                          |
|-------------------------------|--------------------------------------------------------------------------------|
| Lean file                     | <code>VirtualRoots.lean</code>                                                 |
| Public theorems / definitions | 16 / 2                                                                         |
| Axioms (headline theorems)    | <code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code> |
| <code>sorry</code> count      | 0                                                                              |
| Toolchain                     | Lean 4 (godsil pin v4.30) over Mathlib                                         |

Table 2: The development measured. Build: `lake env lean VirtualRoots.lean`, exit 0.

## 8 The boundary of the contribution

*What is proved.* The structural theory of the  $\mathcal{R}_d$  virtual roots, sorry-free: existence and the count ( $= \deg p$ ), that they are sorted and lie in the bracketing interval, that  $\mathcal{R}_d$  returns an actual root wherever  $p$  changes sign, and the Rolle interlacing. These are the results of González-Vega–Lombardi–Mahé [1] in their structural part.

*What is not proved, and is the explicit sequel.* The exact Budan–Fourier count—that the number of virtual roots in  $(a, b]$  equals  $V(a) - V(b)$  [2]—is *not* formalized here. It is not a missing line; it fuses two independent recursive constructions, the  $\mathcal{R}_d$  minimiser of this file and the sign-variation function `fourierVar` of the Budan–Fourier formalization [18], and it is a development in its own right. Stating it as done would be dishonest; it is named here as the next milestone, with the two endpoints (the interlacing here, the inequality there) now both in place.

*Scope of the construction.* The virtual roots are built on a fixed bracketing interval  $[lo, hi]$ . The global object on all of  $\mathbb{R}$  is recovered by taking  $lo, hi$  beyond every root of the derivative tower—a Cauchy bound, available in Mathlib (`Polynomial.cauchyBound`)—but that extension is not carried out in this file.

*Novelty.* To the best of my knowledge, virtual roots had not been formalized in any interactive theorem prover before this. The claim rests on searches in June 2026 of: Mathlib (no virtual roots; it has Descartes via `Polynomial.signVariations`); the Coq `mathcomp-real-closed` library (actual roots and the Thom encoding, not virtual roots); the Isabelle AFP, whose `Budan_Fourier` entry formalizes the inequality with multiplicity but not the virtual-root object; and HOL Light (Sturm and Descartes, not virtual roots). I make this claim at the level of the named object; I cannot exclude an unpublished or differently-named development, and I would welcome a pointer.

*Authorship.* The mathematics is classical, due to the authors cited. The formalization was carried out with an AI assistant; the Lean kernel certifies the proofs, and the scope statements above are mine.

## 9 Reflective questions, for the next bead

A string of beads is meant to be continued. The worth of certifying this structural theory is measured by what it lets one ask next, and ask more sharply. I close with the questions I would rather carry forward than pretend to have settled—each a place where this work stops short, and each, I think, a rung up from it rather than merely a loose end.

1. *From two constructions to one count.* The headline equality  $\#\{\text{virtual roots in } (a, b]\} = V(a) - V(b)$  is the sequel, and there are two honest routes to it. One *bridges* the construction here—the  $\mathcal{R}_d$  minimiser—to the sign-variation function `fourierVar` of the companion work [18], turning that paper’s inequality [2, 1] into an equality. The other *defines* virtual roots a second time, as the jumps of `fourierVar`, where the count is almost definitional, and then proves the two definitions agree. The interlacing proved here is free in the first definition; the count is free in the second; **the equivalence of the two definitions is the real theorem**. Which route is shorter in Lean is itself the question.
2. *Continuity, the other half of their reason for being.* Virtual roots vary *continuously* with the coefficients—this is half of why González-Vega–Lombardi–Mahé introduced them [1]—and continuity is not touched here. A formalization would have to make  $\mathcal{R}_d$ , a chosen minimiser, depend continuously on  $p$ ; the monotonicity bricks that pin the minimiser down are the natural starting point, but the statement is genuinely analytic.

3. *One interlacing, three faces.* The Rolle interlacing places virtual roots beside the families that run through the real-rootedness literature: the roots of a polynomial and its derivative (classical Rolle [6]), Newton’s inequalities [20], and the Heilmann–Lieb matching polynomial. Eisermann’s abstract *Sturm chains* [12] already axiomatize one such interlacing structure. Is there a single Lean notion of “interlacing family” that subsumes all of these—and where does the virtual-root family, defined even when no real roots exist, sit inside it? That is the bead I would most like to carry forward.

## References

- [1] L. González-Vega, H. Lombardi, L. Mahé, Virtual roots of real polynomials, *J. Pure Appl. Algebra* **124** (1998), no. 1–3, 147–166.
- [2] M. Coste, T. Lajous-Loeza, H. Lombardi, M.-F. Roy, Generalized Budan–Fourier theorem and virtual roots, *J. Complexity* **21** (2005), no. 4, 479–486.
- [3] R. Descartes, *La Géométrie* (1637); the rule of signs bounds the number of positive roots by the number of sign changes of the coefficients.
- [4] F. D. Budan de Boislaurent, *Nouvelle méthode pour la résolution des équations numériques*, Paris (1807; 2nd ed. 1822).
- [5] J. Fourier, *Analyse des équations déterminées*, Firmin Didot, Paris (posthumous, 1831).
- [6] M. Rolle, *Traité d’algèbre* (1690); between two consecutive real roots of a differentiable function lies a root of its derivative.
- [7] C. Sturm, Mémoire sur la résolution des équations numériques, *Mémoires présentés par divers savants à l’Académie Royale des Sciences* **6** (1835) 271–318.
- [8] A. G. Akritas, Reflections on a pair of theorems by Budan and Fourier, *Mathematics Magazine* **55** (1982) 292–298.
- [9] G. E. Collins, A. G. Akritas, Polynomial real root isolation using Descartes’ rule of signs, in *Proc. SYMSAC ’76*, ACM, 1976, 272–275.
- [10] A. Alesina, M. Galuzzi, A new proof of Vincent’s theorem, *L’Enseignement Mathématique* **44** (1998) 219–256.
- [11] S. Basu, R. Pollack, M.-F. Roy, *Algorithms in Real Algebraic Geometry*, 2nd ed., Algorithms and Computation in Mathematics **10**, Springer, 2006 (sign-variation theory, Ch. 2, and virtual roots).
- [12] M. Eisermann, The fundamental theorem of algebra made effective: an elementary real-algebraic proof via Sturm chains, *Amer. Math. Monthly* **119** (2012), no. 9, 715–752 (abstract “Sturm chains” and the Cauchy index).
- [13] W. Li, L. C. Paulson, Counting polynomial roots in Isabelle/HOL: a formal proof of the Budan–Fourier theorem, in *Proc. CPP 2019*, ACM, 2019, 52–64; Archive of Formal Proofs entry “The Budan–Fourier Theorem” (W. Li, 2018), [https://isa-afp.org/entries/Budan\\_Fourier.html](https://isa-afp.org/entries/Budan_Fourier.html) (formalizes the inequality with multiplicity, not the virtual-root object).
- [14] C. Cohen, Construction of real algebraic numbers in Coq, in *Interactive Theorem Proving (ITP 2012)*, LNCS **7406**, Springer, 2012, 67–82 (basis of `mathcomp-real-closed`: actual roots and the Thom encoding, not virtual roots).
- [15] M. Eberl, Sturm’s Theorem, *Archive of Formal Proofs* (2014), [https://isa-afp.org/entries/Sturm\\_Sequences.html](https://isa-afp.org/entries/Sturm_Sequences.html).



- [16] The mathlib Community, The Lean mathematical library, in *Certified Programs and Proofs (CPP 2020)*, ACM, 2020, 367–381.
- [17] L. de Moura, S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction (CADE 28)*, LNCS **12699**, Springer, 2021, 625–635.
- [18] C. Marín, *Counting with the Derivative Tower: the Budan–Fourier Theorem, Machine-Checked in Lean 4* (2026, companion formalization; `BudanFourier.lean`).
- [19] C. Marín, *The Staircase of Signs: Sturm’s Root-Counting Theorem, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20707348.
- [20] C. Marín, *The Inward Bow of a Real-Rooted Polynomial: Newton’s Inequalities, Machine-Checked in Lean 4*, Zenodo, 2026, doi:10.5281/zenodo.20693064.